# DeepTracer 功能文档

## 1. 项目概述

DeepTracer 是一个面向 Python 代码理解、运行观察和改进建议的代码分析工具。项目提供命令行入口和本地 Web 工作台两种使用方式，核心能力围绕“输入 Python 代码 -> 执行本地分析 -> 生成结构、性能、内存与优化建议 -> 支持继续追问”展开。

DeepTracer 的功能设计重点不是替代 IDE、静态检查器或通用大模型问答，而是把一段 Python 代码转化为可阅读、可定位、可继续讨论的分析结果。它适合用于代码学习、代码讲解、初步性能定位、重构前理解、课堂演示和轻量级代码诊断。

当前项目可运行能力由以下几部分组成：

- 命令行分析入口：`deeptracer`
- 本地 Web 工作台：`deeptracer-web`
- FastAPI 分析服务
- LangGraph 分析流水线
- 本地 AST、性能、内存分析器
- 可选的 OpenAI 兼容模型增强分析
- 基于当前分析结果的流式问答
- 本地 Python Tutor 执行过程可视化

## 2. 目标用户与使用场景

### 2.1 目标用户

DeepTracer 面向需要理解 Python 代码的人群，包括：

- Python 初学者：需要知道代码从哪里开始看、函数职责是什么、为什么某段代码值得关注。
- 教学者或助教：需要把代码结构、运行行为和优化建议以更直观的方式展示给学习者。

- 开发者：需要对单个 Python 文件或代码片段做快速结构分析和轻量级运行诊断。
- 项目分析人员：需要从功能层面理解工具能力、页面组成、输入输出和系统边界。

## 2.2 典型场景

- 对一个 `.py` 文件进行快速分析，查看代码行数、重点函数、运行耗时和优化建议。
- 在 Web 页面中粘贴或载入 Python 代码，观察 AST 结构、性能热点和内存摘要。
- 对分析结果继续提问，例如“为什么这个函数是重点函数”“哪里可能慢”“下一步先改哪里”。
- 通过 Python Tutor 可视化观察代码执行过程，辅助理解变量变化和控制流。
- 在没有配置模型 API 的环境中，以本地规则和本地分析结果完成基础分析。

# 3. 功能入口

## 3.1 命令行入口

安装项目后可以使用 `deeptracer` 命令分析 Python 文件：

```
deeptracer path/to/file.py
```

命令行入口会完成以下工作：

- 校验输入路径是否存在。
- 校验输入文件是否为 `.py` 文件。
- 调用当前分析服务分析目标文件。
- 在终端输出摘要信息，包括分析对象、代码行数、重点函数、执行耗时和前三条建议标题。
- 如果目标代码运行时报错，会在终端摘要中展示运行错误。

命令行入口适合快速检查单个 Python 文件，不提供 Web 页面中的图形化结果、交互式问答和执行流程可视化。

### 3.2 Web 工作台入口

安装项目后可以使用 `deeptracer-web` 启动本地 Web 服务：

```
deeptracer-web
```

服务默认监听：

```
http://127.0.0.1:8000
```

Web 工作台提供完整交互体验，包括文件载入、代码编辑、分析结果展示、可视化、优化建议和开放式答疑。

## 4. Web 工作台功能

### 4.1 首页与代码载入

Web 首页提供三种代码进入方式：

- 选择本地 Python 文件。
- 将 `.py` 文件拖放到页面。
- 载入内置示例代码。

文件载入后，前端会读取文件内容并进入代码页面。当前前端只接受 `.py` 后缀文件；如果选择其他文件类型，会直接提示用户选择 Python 文件。

### 4.2 代码查看与编辑

代码页面包含源代码区域和执行流程区域。源代码区域提供：

- 带行号的代码查看。
- 当前文件名显示。
- 当前代码行数显示。
- 清空代码。
- 重新加载当前代码内容。
- 复制代码到剪贴板。
- 分析前后编辑。

- 保存编辑后展示简易 Diff 对比。

编辑功能用于在页面内临时调整代码并重新分析。保存编辑后，页面会用红绿行标记展示删除和新增内容，帮助用户理解本次改动。

## 4.3 代码分析触发

点击“分析代码”后，前端会向后端提交当前代码内容和显示名称。后端返回结构化分析结果后，页面会更新：

- 顶部核心指标。
- 概览、结构、性能、内存标签页。
- AST 和性能可视化区域。
- 优化建议列表和详情。
- 运行模式提示。
- 后续问答上下文。

分析期间页面会显示加载状态，避免用户误以为请求无响应。

## 4.4 执行流程可视化

代码页面右侧提供“执行流程”区域。它基于本地集成的 Python Tutor 运行时生成执行 trace，并在页面中渲染代码执行过程。

该功能支持：

- 分析完成后自动尝试加载执行可视化。
- 点击“刷新”重新生成当前代码的执行流程。
- 展示变量、栈帧和执行步骤等 Python Tutor 可视化内容。

该功能依赖项目内 `vendor/pathrise-python-tutor/v5-unity` 目录。如果本地 Python Tutor 资源不可用，服务会通过状态接口返回不可用状态，相关可视化不会作为必选能力执行。

## 4.5 分析结果页

分析结果页由多个区域组成。

### 4.5.1 核心指标

核心指标来自当前分析对象，主要包括：

- 分析对象。
- 代码行数。
- 重点函数。
- 执行耗时。

这些指标用于帮助用户先获得全局判断，再决定是否深入结构、性能或内存详情。

### 4.5.2 概览

概览标签页展示当前代码的综合判断，包括：

- 结构复杂度。
- 热点集中度。
- 内存压力。
- 修改空间。
- 分析对象与常见语法类型。
- 分析是否完整完成。

概览用于回答“这段代码整体情况如何”“哪里最值得先看”。

### 4.5.3 结构分析

结构分析基于 Python AST，展示：

- 代码行数。
- 函数数量。
- 重点函数。
- 函数复杂度排序。

- 输出语句数量。
- 结构阅读建议。

重点函数的判断依据包括函数内部控制流数量、函数跨度和复杂度。该模块适合用于初学者理解代码组织方式，也适合用于重构前定位复杂函数。

### 4.5.4 性能分析

性能分析基于本地执行结果和 `cProfile` 统计，展示：

- 本次执行耗时。
- 热点函数。
- 可见性能样本数量。
- 函数级耗时占比。
- 循环中的可疑重复调用。
- 运行异常信息。

性能结果来自真实执行，而不是单纯静态推断。如果代码在执行时异常，系统仍会尽量保留已经收集到的耗时、热点和错误信息。

### 4.5.5 内存分析

内存分析基于 `tracemalloc`，展示：

- 峰值内存。
- 热点位置数量。
- 运行状态。
- 内存热点行。
- 内存使用解释。

该模块用于帮助用户判断是否存在明显大对象、长时间持有对象或内存热点。它不是完整的内存泄漏检测器，而是轻量级运行摘要。

### 4.5.6 可视化分析

可视化区域包含两个标签：

- AST 语法树：使用本地 AST 分析器生成 HTML 可视化。
- 性能火焰图：使用性能分析器生成 HTML 报告。

如果可视化生成失败，后端会返回空内容，页面仍保留其他文字和结构化分析结果。

### 4.5.7 优化建议

优化建议模块展示一到三条可操作建议。每条建议包含：

- 标题。
- 优先级。
- 置信度。
- 影响范围。
- 风险标签。
- 说明。
- 被选中的原因。
- 相关代码锚点。
- 不处理的风险。
- 下一步建议。
- Diff 风格的修改预览。

建议生成会优先结合本地分析结果，包括复杂函数、循环内重复调用、调试输出和内存峰值等。配置模型 API 后，模型驱动的 Refactor Agent 可以在本地建议基础上生成更自然的解释和建议。

## 4.6 开放式答疑

分析结果页提供开放式答疑区域，用户可以围绕当前代码继续提问。

问答功能的特点：

- 以当前分析结果和当前代码为上下文。
- 保留最近的对话消息。

- 通过 Server-Sent Events 流式返回回答。
- 支持停止当前回答。
- 支持 Markdown 段落、列表、表格和代码块渲染。
- 支持复制回答中的代码块。
- 在未配置模型 API 时，使用本地保底回答。

问答服务会构建 grounding bundle，将当前代码、分析摘要、建议和代码锚点注入上下文，要求回答明确对齐到真实函数名、变量名或代码片段，降低泛化和编造风险。

### 4.7 主题与布局

前端提供基础体验功能：

- 三种主题：`mist`、`paper`、`night`。
- 首页、代码页、分析结果页三页式导航。
- 代码区和执行流程区之间的拖拽宽度调节。
- 响应式布局约束。
- 加载动画和状态提示。

这些能力服务于可读性和演示体验，不影响后端分析结果。

## 5. 后端分析能力

### 5.1 分析流水线

后端使用 LangGraph 组织分析流程。当前流水线按顺序执行：

1. `local_tools`：运行本地分析，包括 AST、性能和内存分析。
2. `core_agents`：并行运行结构、性能和内存三个解释型 agent。
3. `refactor`：生成重构或优化建议。
4. `teaching`：生成更适合初学者阅读的讲解。
5. `response`：合并所有结果，构造前端响应。

其中 `core_agents` 阶段使用线程池并行执行结构、性能和内存 agent，以减少整体等待时间。

## 5.2 本地 AST 分析

AST 分析会解析 Python 源码并提取：

- 函数定义。
- 异步函数定义。
- 函数起止行。
- 函数跨度。
- 控制流节点数量。
- 常见语法节点统计。
- `print` 调用位置。
- 循环内部的函数调用。

AST 分析结果用于结构页、概览评分、建议生成和问答代码锚点。

## 5.3 本地性能分析

性能分析会在受控上下文中执行目标代码，并使用 `cProfile` 收集函数调用统计。它会返回：

- 总执行耗时。
- 当前文件内的函数调用样本。
- 调用次数。
- 总耗时。
- 自身耗时。
- 热点函数排序。
- 标准输出摘要。
- 运行错误摘要。

执行时会临时调整工作目录和 `sys.path`，以便目标文件能按自身目录运行。执行结束后会恢复原工作目录和导入路径。

## 5.4 本地内存分析

内存分析使用 `tracemalloc` 跟踪目标代码运行期间的内存情况。它会返回：

- 峰值内存。

- 当前文件内的热点内存行。
- 每个热点行的内存占比。
- 内存占用大小。

该功能适合发现明显内存热点，但不承诺提供长时间运行服务的完整泄漏诊断。

### 5.5 运行异常处理

如果被分析代码运行时报错，系统会：

- 捕获异常类型和异常信息。
- 保留标准输出中已经产生的内容。
- 尽量保留已收集的性能和内存样本。
- 在分析结果中标记运行状态。
- 在建议和问答中继续使用可确定的结构信息。

这使得 DeepTracer 可以分析“结构可解析但运行失败”的代码，而不是直接丢弃全部结果。

### 5.6 可视化生成

后端会尝试生成两类 HTML 可视化：

- AST 可视化：由 `AstAnalyer` 生成。
- 性能报告：由 `SpeedAnalyzer` 生成。

可视化生成失败不会中断主分析流程；系统会返回空可视化内容并保留文字分析、指标和建议。

## 6. 模型增强能力

### 6.1 模型配置

DeepTracer 支持可选的 OpenAI 兼容模型配置。运行时会读取环境变量或本地配置文件，主要配置项包括：

- `OPENAI_API_KEY`
- `OPENAI_BASE_URL`

- `OPENAI_MODEL`
- `DEEPTRACER_MODEL_PROVIDER`
- `DEEPTRACER_MODEL_NAME`
- `DEEPTRACER_MODEL_TEMPERATURE`
- `DEEPTRACER_MODEL_TIMEOUT_SECONDS`
- `DEEPTRACER_MODEL_MAX_RETRIES`

当前模型提供方逻辑只支持 `openai`。如果未配置 API key，系统会进入本地回退模式。

## 6.2 多智能体分析

配置模型后，DeepTracer 会在本地分析结果基础上运行五类 agent：

- Structure Agent：解释代码结构，指出优先阅读的函数。
- Performance Agent：解释性能热点。
- Memory Agent：解释内存情况。
- Refactor Agent：综合结构、性能和内存结果生成建议。
- Teaching Agent：生成初学者友好的整体说明、提示和下一步。

所有 agent 都有本地回退结果。如果模型不可用、调用失败或输出不符合结构化约束，系统会回退到本地分析内容，保证页面仍可用。

## 6.3 本地回退模式

未配置模型 API 时，DeepTracer 仍支持：

- 文件和代码输入。
- AST 分析。
- 性能分析。
- 内存分析。
- 可视化生成。
- 本地规则建议。
- 本地模板式问答。
- Python Tutor 执行流程可视化。

回退模式会在页面运行提示中明确展示，避免用户误认为所有解释都来自模型。

# 7. API 功能

## 7.1 `GET /`

返回 Web 工作台首页。

## 7.2 `POST /api/analyze`

分析 Python 代码或 Python 文件路径。

请求可包含：

- `code`：内联 Python 源码。
- `path`：Python 文件路径。
- `displayName`：页面展示名称。
- `taskId`：已有分析任务 ID。
- `analysis`：已有分析快照。

返回内容包括：

- `heroMetrics`
- `analysisMap`
- `suggestions`
- `stages`
- `visualizations`
- `teaching`
- `teachingPanels`
- `analysisSources`
- `runtimeBanner`
- `meta`

### 7.3 `POST /api/chat/stream`

基于当前代码和分析结果进行流式问答。

请求需要包含：

- `code` 或 `path`
- `messages`
- 可选的 `displayName`
- 可选的 `taskId`
- 可选的 `analysis`

响应类型为 `text/event-stream`，事件包括：

- `meta`
- `chunk`
- `done`
- `error`

### 7.4 `GET /api/tutor/status`

检查本地 Python Tutor 是否可用。

返回：

- `available`
- `embedUrl`

### 7.5 `POST /api/tutor/trace`

执行 Python Tutor trace 生成。

请求包含：

- `code`
- 可选 `rawInput`

- 可选 `options`

返回 Python Tutor 可视化所需的 trace payload。

### 7.6 Python Tutor 兼容路由

Web 服务提供本地 Python Tutor iframe 和执行路由：

- `GET /tutor/iframe-embed.html`
- `GET /tutor/web_exec_py3.py`

这些路由服务于当前 Web 页面内嵌执行可视化。

## 8. 数据与状态管理

### 8.1 分析任务 ID

每次完整分析会生成 `taskId`，用于将后续问答、分析快照和缓存结果关联到同一段代码。

### 8.2 分析缓存

后端维护一个内存级 LRU 缓存，最多保存 24 条分析记录。缓存内容包括：

- 分析响应 payload。
- 对应源码。

缓存用于减少重复分析，并为问答功能提供稳定上下文。缓存不持久化到数据库，服务重启后会清空。

### 8.3 Grounding Bundle

问答前会构造 grounding bundle，包含：

- 当前显示名称。
- 当前源码。
- 概览、结构、性能、内存摘要。
- 前三条建议。

- 函数、变量、调用等代码锚点。

该结构用于让回答尽量围绕当前真实代码展开。

# 9. 输入、输出与限制

## 9.1 支持输入

当前真正支持的输入包括：

- `.py` 文件路径。
- Web 页面中读取的 `.py` 文件内容。
- Web 页面内联 Python 源码。

## 9.2 输出形式

当前输出包括：

- CLI 文本摘要。
- Web 指标卡片。
- Web 标签页分析结果。
- AST 可视化 HTML。
- 性能报告 HTML。
- 优化建议与 Diff 预览。
- SSE 流式问答。
- Python Tutor trace 和前端可视化。

## 9.3 功能限制

当前功能边界如下：

- 只面向 Python 代码分析。
- CLI 只接受 `.py` 文件路径。
- Web 文件选择只接受 `.py` 文件。
- 运行分析会执行目标代码，因此不适合直接分析不可信代码。

- 性能和内存结果取决于本地运行环境、输入数据规模和代码是否能实际执行。
- 内存分析是轻量级摘要，不是完整生产级泄漏诊断。
- 模型能力是可选增强，不是基础分析的前置条件。
- 当前模型提供方逻辑只支持 OpenAI 兼容调用。
- Web 前端是原生 HTML、CSS 和 JavaScript，不包含独立前端构建系统。

## 10. 功能价值总结

DeepTracer 当前真正可运行的功能可以概括为：

- 用本地服务快速分析 Python 文件或代码片段。
- 将代码结构、性能和内存信息拆成可读的页面模块。
- 给出带原因、风险和代码锚点的优化建议。
- 在模型可用时提供更自然的多 Agent 解释。
- 在模型不可用时保持本地分析和基础问答可用。
- 通过 Python Tutor 帮助用户观察代码执行过程。
- 通过 Web 工作台把分析、可视化、建议和追问组织在同一个使用流程中。

DeepTracer 的核心定位是“围绕单段 Python 代码的理解型分析工作台”。它把本地可验证的运行数据作为基础，再以可选模型解释增强可读性，使没有参与项目代码工作的人也能从功能层面理解项目的输入、处理过程、输出结果和使用边界。